

TropicalGT IV: Tropical Oracle Trajectory Classifiers for Behavior, Reasoning Quality, and Bits-per-byte Control

Amelie Schreiber

June 2026

Abstract

TropicalGT I introduced a graph-token reasoning model whose attention core can operate in the max-plus or min-plus semiring, exposing active supports, polyhedral margins, and dynamic-programming certificates while preserving bits-per-byte accountability. TropicalGT II lifted this local mechanism to iterative reasoning trajectories: standardized graph-of-thought packets, tropicalized hidden states, trajectory amoebas, patchworked reasoning charts, filtered simplicial persistence complexes, and ergodic cell summaries. TropicalGT III moved the same mathematics to the external substrate of modern agents: model-context-protocol traces, markdown memory, knowledge graphs, long-context windows, graph-structured vector databases, projected retrieval heads, tropical quantization, and context packing up to multi-million-token budgets.

This fourth paper adds an oracle layer. The goal is to train either a compact additional head or a full oracle transformer that classifies partial and complete reasoning trajectories by properties that matter for deployment: correctness, verifier agreement, evidence grounding, proof rigor, tool discipline, compliance, safety, refusal appropriateness, tone, personality corridor, verbosity, calibration, efficiency, context-token utility, retrieval hygiene, memory freshness, and held-out bits-per-byte utility. The oracle is not a post-hoc sentiment tagger. It is a structured tropical classifier over graph-token trajectories and context graphs. Its logits are piecewise-linear tropical polynomials; its label space is a property poset equipped with incompatibility complexes; its local judgments form a sheaf over trajectory windows; its control actions are computed by min-plus dynamic programming; and its deployment value is measured by downstream model improvement rather than oracle accuracy alone.

We develop definitions, theorem/proof statements, training losses, pseudocode, data schemas, ablations, and implementation protocols. We prove equivariance, proper-scoring, side-information, tropical-margin, quantization-stability, Cech-gluing, Viterbi-control, monotone-policy-improvement, and bits-per-byte-gated promotion results under explicit finite or compact assumptions. The paper is pragmatic: every classifier signal must either improve model behavior, lower error, reduce context waste, improve safety/compliance, improve style control, or produce a positive held-out bits-per-byte tradeoff after artifact and runtime costs.

Contents

1	Introduction	5
2	Continuity with TropicalGT I–III and the mathematical sources	6
2.1	Tropical attention as the local primitive	6
2.2	Tropical geometry as global organization	6
2.3	TropicalGT III and the external context substrate	7
2.4	The fourth iteration	7

3	Trajectory objects and property spaces	8
3.1	Trajectory packets	8
3.2	Property taxonomy	8
4	Tropical oracle architectures	9
4.1	Compact oracle head	9
4.2	Full oracle transformer	10
4.3	Hybrid design	10
5	Tropical property fans and margin geometry	10
5.1	Property fans	10
5.2	Tropical decision stability	11
6	Structured oracle losses	12
6.1	Proper scoring and calibration	12
6.2	Poset isotonicity	12
6.3	Incompatibility penalties	13
6.4	Tropical margin and active-support losses	13
6.5	Sheaf gluing over trajectory windows	13
6.6	Composite objective	14
7	Oracle-guided training and control	14
7.1	From labels to training actions	14
7.2	Side information and bits-per-byte	14
7.3	Bayes thresholds for risk properties	15
7.4	Min-plus behavior control	15
7.5	Oracle-guided policy improvement	15
8	Label acquisition and weak supervision	16
8.1	Label sources	16
8.2	Weak-label model	16
8.3	Synthetic perturbation examples	16
9	Training paradigms	17
9.1	Default staged recipe	17
9.2	Pseudocode: oracle head training	17
9.3	Pseudocode: oracle-assisted base-model training	18
9.4	Pseudocode: test-time behavior projection	18
10	Applications to reasoning quality and behavior control	18
10.1	Accuracy and proof rigor	18
10.2	Grounding and citation support	18
10.3	Compliance and refusal appropriateness	19
10.4	Tone and personality corridor	19
10.5	Efficiency and context-token utility	19
10.6	Tool and MCP discipline	19

11 Evaluation metrics and ablation design	19
11.1 Oracle metrics	19
11.2 Downstream model metrics	19
11.3 Ablation ladder	20
12 Implementation details	20
12.1 Trajectory record schema	20
12.2 Feature extraction	21
12.3 Recommended hyperparameters	21
12.4 Reproducibility protocol	21
13 Theoretical guarantees for deployment gates	21
13.1 Equivariance	21
13.2 Blockwise long-context oracle aggregation	22
13.3 Oracle promotion gate	22
14 Objective catalogue	22
15 Worked examples	29
15.1 Mathematical proof trajectory	29
15.2 MCP tool trajectory	30
15.3 Long-context memory trajectory	30
15.4 Tone/personality control trajectory	30
16 Full oracle transformer methodology	30
16.1 Packet-token construction	30
16.2 Windowed and hierarchical inference	31
16.3 Universal approximation on bounded trajectory families	31
17 Benchmark suite and data generation	32
17.1 Benchmark families	32
17.2 Counterfactual trajectory generation	32
17.3 Dataset split discipline	33
18 Deployment patterns	33
18.1 Teacher-side oracle	33
18.2 Shadow oracle	33
18.3 Runtime gate	33
18.4 Exported behavior adapter	33
18.5 Audit log	33
19 Additional tropical objectives for TropicalGT IV	33
20 Safety and limitations	35
21 Reproducibility checklist	35
22 Conclusion	36

A	Additional proof details	36
A.1	Nonexpansiveness of max-plus logits	36
A.2	Stanley-Reisner-style incompatibility and gradients	36
A.3	Why tropical aggregation is useful for qualitative labels	37
B	Initial experimental plan	37
C	Bibliography	37

1 Introduction

The first three TropicalGT papers developed a structured training program for graph-token reasoning agents. The first paper used tropical attention to expose max-plus supports and dynamic-programming structure. The second paper treated repeated reasoning as a dynamical system of tropicalized packets and persistent complexes. The third paper treated MCP traces, markdown memory, knowledge graphs, vector databases, and long-context windows as an external tropical context substrate. These papers answer three questions: how should a reasoning step compute, how should a reasoning trajectory evolve, and how should an agent retrieve and assemble context.

A fourth question remains: how should the system know which trajectories are good? A next-token loss evaluates a final byte stream. A verifier checks only the task slice for which it was designed. A safety classifier often sees only a final answer. A retrieval metric may measure only top-k relevance. None of these alone says whether the entire trajectory was accurate, grounded, efficient, compliant, calibrated, safe, properly toned, memory-hygienic, and context-efficient. For that we need an oracle trajectory classifier.

The oracle problem. Let a base agent produce a trajectory

$$\tau = (R_0, R_1, \dots, R_T, y), \quad (1.1)$$

where each packet R_t contains visible text, hidden summaries, graph-of-thought state, retrieved evidence, MCP or tool records, markdown-memory nodes, knowledge-graph neighborhoods, active tropical supports, verifier fields, and optional persistence or certificate data. The oracle maps

$$\mathcal{O}_\omega : \tau \mapsto \hat{p}_\omega(\cdot \mid \tau) \quad (1.2)$$

where the output is a calibrated distribution or score vector over trajectory properties. Properties may be binary, ordinal, continuous, or structured. Examples include `correct`, `grounded`, `unsafe-compliance`, `tool-relevant`, `memory-stale`, `tone-drift`, `too-verbose`, `proof-gap`, `context-waste`, and `answer-ready`.

Why tropical? Tropical attention is naturally suited to trajectory classification for three reasons. First, many trajectory labels are dominated by worst-case or best-case events: one critical hallucinated citation, one unsafe tool call, one decisive proof gap, one high-quality evidence path, or one context block that makes the answer possible. Max-plus and min-plus aggregation represent these extrema directly. Second, tropical logits are piecewise-linear and have active supports, so the oracle can explain a label by the particular window, evidence block, tool event, or hidden cell that determined it. Third, tropical margins make compression safe: if a label decision has a large margin, quantization or projection can preserve it.

The central thesis. A trajectory oracle is useful only if it changes model training or inference in a beneficial way. It can filter low-quality data, weight supervised losses, construct preference pairs, shape GFlowNet rewards, reject bad graph-of-thought branches, gate MCP tool calls, prune stale memories, steer tone corridors, or stop long-context packing. But its value is not its standalone F1 score. Its value is downstream utility:

$$U(\mathcal{O}) = \Delta \text{bits-per-byte}_{\text{heldout}} + \lambda_A \Delta A + \lambda_R \Delta R + \lambda_C \Delta C - \rho_B \text{bytes}(\mathcal{O}) - \rho_T \text{latency}(\mathcal{O}), \quad (1.3)$$

where A is task accuracy, R is reasoning/retrieval quality, and C is control reliability. If $U \leq 0$, the oracle may remain teacher-side but should not be promoted into a deployed artifact.

Contributions. This paper contributes the following.

1. It defines tropical trajectory-property classifiers for partial and complete graph-token reasoning traces.
2. It gives two architectures: a compact oracle head on a frozen or jointly trained backbone, and a full oracle transformer over trajectory packets.
3. It imposes structure on labels through property posets, incompatibility complexes, tropical property fans, local-window sheaves, calibration maps, and min-plus control graphs.
4. It proves core guarantees under explicit assumptions: equivariance, tropical active-support stability, quantization safety, proper scoring, side-information bounds, sheaf gluing, Viterbi-optimal control, and oracle-guided policy improvement.
5. It gives a practical training paradigm: data schema, label acquisition, weak supervision, verifier distillation, human calibration, synthetic perturbation generation, loss functions, pseudocode, ablations, deployment profiles, and reproducibility checklists.
6. It provides an objective catalogue for improving reasoning accuracy, evidence grounding, behavior control, tone/personality stability, safety/compliance, long-context efficiency, and held-out bits-per-byte.

2 Continuity with TropicalGT I–III and the mathematical sources

2.1 Tropical attention as the local primitive

The NeurIPS 2025 Tropical Attention paper introduces multi-head tropical attention as a max-plus attention mechanism for neural algorithmic reasoning. It proves approximation results for tropical circuits and realizes tropical transitive closure through composition, explaining why the induced polyhedral decision boundaries remain sharp and scale-invariant rather than being smoothed by softmax [1, 2]. The released implementation includes a standalone tropical attention kernel and combinatorial algorithm tasks [3]. TropicalGT I used that mechanism as a graph-token primitive.

A max-plus head computes

$$Y_{ic} = \max_j \{S_{ij} + V_{jc}\}. \quad (2.1)$$

For classification, replace the output coordinate c by a property index p . The active set

$$A_{ip} = \arg \max_j \{S_{ij} + V_{jp}\} \quad (2.2)$$

is a finite explanation: it identifies the token, window, evidence block, or certificate branch controlling the property score. This is the central reason to use tropical structure for oracle labels. It provides classifier explanations at the level of trajectory events rather than only feature saliency.

2.2 Tropical geometry as global organization

The attached tropical algebraic geometry resource introduces amoebas as logarithmic images of algebraic varieties, studies limits of rescaled amoebas, constructs non-Archimedean amoebas by valuations, defines the tropical semifield, defines tropical polynomials as maxima of affine functions, and defines tropical hypersurfaces as corner loci. It also develops tropical curves with integer affine structure, balancing, Viro patchworking, and correspondence-style enumerative results. These concepts gave TropicalGT II its language of trajectory amoebas, patchworked reasoning charts, balanced evidence flow, and ergodic cell occupation.

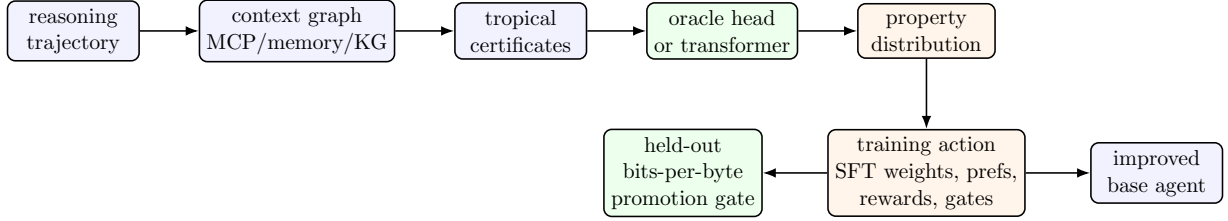


Figure 1: TropicalGT IV system view. The oracle reads trajectory packets, context events, and tropical certificates; predicts structured trajectory properties; and uses those predictions only through gated training or inference actions.

For TropicalGT IV, the relevant analogue is the *property fan*. A tropical oracle logit is

$$g_p(\tau) = \max_{r \leq R_p} \{b_{pr} + \langle a_{pr}, \phi(\tau) \rangle\}, \quad (2.3)$$

where $\phi(\tau)$ is a trajectory embedding or pooled packet representation. The regions on which a single affine branch controls g_p form a polyhedral complex. Boundaries between branches are tropical hypersurfaces. When the class decision changes, the trajectory crosses a wall of this complex. That wall should correspond to an interpretable event: evidence appeared, a proof gap closed, an unsafe request entered, a citation became stale, or a tone corridor was violated.

2.3 TropicalGT III and the external context substrate

TropicalGT III extended the theory from internal hidden trajectories to MCP traces, markdown memory, knowledge graphs, vector databases, graph-structured retrieval, projected retrieval heads, quantization, and long-context windows. The external substrate was modeled as a valuated context graph whose nodes carry dense embeddings, tropical valuation vectors, quantized codes, provenance/security metadata, adjacency, and finite certificates. Retrieval was treated as a tropical graph operation rather than a flat nearest-neighbor lookup. The official MCP specification frames MCP as a protocol connecting LLM applications to external data sources and tools, and describes server-exposed tools with schemas and invocation metadata [13, 14]. TropicalGT III turns those protocol events into graph nodes; TropicalGT IV classifies the trajectories that use them.

2.4 The fourth iteration

TropicalGT IV is therefore the classifier/control layer over the previous three levels. The flow is:

$$\begin{aligned} &\text{tropical attention} \rightarrow \text{trajectory dynamics} \rightarrow \text{context retrieval} \\ &\rightarrow \text{oracle trajectory typing} \rightarrow \text{improved training/control}. \end{aligned} \quad (2.4)$$

The oracle should see all relevant finite summaries but should not force private chain-of-thought exposure. It may consume hidden summaries, active supports, evidence ids, verifier labels, tool schemas, context usage statistics, and final outputs. User-visible reporting should expose concise rationales, citations, uncertainty, and verifier status, not raw hidden reasoning traces.

3 Trajectory objects and property spaces

3.1 Trajectory packets

Definition 3.1 (Reasoning packet). A reasoning packet is a tuple

$$R_t = (G_t, H_t, X_t, C_t, E_t, V_t, A_t, B_t, M_t, U_t), \quad (3.1)$$

where G_t is a typed graph-of-thought state, $H_t \in \mathbb{R}^{n_t \times d}$ is a hidden summary table, X_t is visible text or a text pointer, $C_t \subseteq \mathfrak{C}$ is the retrieved context subgraph, E_t is evidence/provenance metadata, V_t is verifier state, A_t is a tropical active-support certificate, B_t is an optional persistence or barcode summary, M_t is MCP/tool metadata, and U_t records behavior-control coordinates such as tone, verbosity, uncertainty, and safety-corridor features.

Definition 3.2 (Trajectory). A trajectory is a finite sequence

$$\tau = (R_0, R_1, \dots, R_T, y) \quad (3.2)$$

with final output y . A partial trajectory is $\tau_{\leq t} = (R_0, \dots, R_t)$. A trajectory is *context-complete* for a task if the answer is expected to be derivable from the prompt, retrieved subgraphs, tool outputs, and generated intermediate states inside τ , subject to allowed inference rules.

This object is intentionally more detailed than a transcript. It makes explicit the hidden, retrieved, and audited pieces that a trajectory oracle may need. In deployed privacy-preserving settings, the oracle can be trained or run on compressed summaries of these fields rather than raw private reasoning text.

3.2 Property taxonomy

Let $\mathcal{P} = \{p_1, \dots, p_m\}$ be a finite property set. We distinguish five families.

Epistemic properties. Correctness, verifier agreement, factual grounding, citation support, uncertainty calibration, proof rigor, contradiction absence, and evidence sufficiency.

Computational properties. Efficiency, context-token utility, number of tool calls, latency, retrieval precision, retrieval closure quality, memory freshness, and bits-per-byte utility.

Behavioral properties. Compliance, refusal appropriateness, safety-boundary respect, safe completion, unsafe compliance risk, and policy consistency.

Stylistic properties. Tone corridor, personality profile stability, verbosity match, directness, uncertainty expression, didactic level, and user-intent adaptation.

Structural properties. Stable active supports, large tropical margins, balanced evidence flow, patchworking gluing, persistence stability, ergodic non-thrashing, and context graph coherence.

Definition 3.3 (Property poset). A property poset is a finite partially ordered set $(\mathcal{P}, \preceq)$ where $p \preceq q$ means that property q logically or operationally implies property p . For example, **fully-grounded** implies **partly-grounded**; **unsafe-compliance** implies **compliance-risk**; and **strict-proof-valid** implies **verifier-agreement** for a proof verifier under specified assumptions.

Definition 3.4 (Incompatibility complex). An incompatibility complex is an abstract simplicial complex $\Delta_{\mathcal{P}} \subseteq 2^{\mathcal{P}}$ whose faces are jointly compatible property sets. Minimal nonfaces $F \notin \Delta_{\mathcal{P}}$ encode forbidden simultaneous labels such as `{fully-grounded, unsupported-citation}`, `{safe-refusal, unsafe-compliance}`, or `{concise-answer, excessively-verbose}`.

Definition 3.5 (Label module). A trajectory label is a vector $y \in [0, 1]^m$, an ordinal vector $o \in \prod_j \{0, \dots, K_j\}$, or a probability distribution over compatible subsets of $\mathcal{P}$. A label module is the graded vector space

$$L = \bigoplus_{p \in \mathcal{P}} L_p \quad (3.3)$$

whose coordinates are grouped by property family, severity, evidence type, and context-window location.

The point of this structure is to prevent label independence from producing incoherent training. A multi-label classifier that predicts both `properly-refuses` and `unsafe-compliance` with high confidence is not merely uncertain; it violates a known compatibility constraint. A classifier that predicts `strict-proof-valid` but not `verifier-agreement` is similarly inconsistent unless the verifier is known to be incomplete.

4 Tropical oracle architectures

4.1 Compact oracle head

A compact oracle head is attached to a base TropicalGT or graph-token model. Let Φ_{θ} be the base model and $s_{\theta}(\tau) \in \mathbb{R}^d$ a pooled trajectory summary. The summary may combine final hidden states, active-support histograms, verifier features, context usage statistics, and compressed evidence IDs:

$$s_{\theta}(\tau) = \text{Pool}(R_0, \dots, R_T). \quad (4.1)$$

For property p , define a tropical logit

$$g_p(\tau) = \max_{r \leq R_p} \{b_{pr} + \langle a_{pr}, s_{\theta}(\tau) \rangle\}. \quad (4.2)$$

The probabilities are obtained by a calibration map, for instance

$$\hat{y}_p = \text{sigmoid}(\alpha_p g_p + \delta_p), \quad (4.3)$$

for binary labels or a softmax over class logits for multiclass labels.

Definition 4.1 (Tropical oracle head). A tropical oracle head of rank R is a collection of piecewise-linear logits

$$g_p(s) = \bigoplus_{r=1}^{R_p} (b_{pr} \odot a_{pr}s) = \max_r \{b_{pr} + \langle a_{pr}, s \rangle\}, \quad (4.4)$$

with active branch

$$A_p(s) = \arg \max_r \{b_{pr} + \langle a_{pr}, s \rangle\}. \quad (4.5)$$

Its explanation code is the tuple $(p, A_p(s), \Delta_p(s), W_p(s))$, where Δ_p is the top-two margin and W_p is a pointer to the highest-contributing trajectory window.

The compact head is the default first experiment. It is cheap, easy to calibrate, and can be exported only if useful. It is appropriate when labels can be predicted from pooled summaries. It will fail when raw evidence spans, local tool schemas, or long-context ordering are decisive.

4.2 Full oracle transformer

A full oracle transformer reads the trajectory as a token sequence or graph. The token types include packet summaries, message spans, retrieved blocks, MCP calls, tool outputs, knowledge-graph nodes, verifier judgments, active-support certificates, and behavior coordinates. Attention can mix ordinary softmax heads with tropical heads. Tropical heads are used for labels governed by extremal evidence: a single unsafe call, the best proof path, worst contradiction, strongest citation, or largest context-waste block.

Definition 4.2 (Oracle transformer). An oracle transformer is

$$\mathcal{O}_\omega = D_\omega \circ \Psi_L \circ \cdots \circ \Psi_1 \circ T_{\text{traj}}, \quad (4.6)$$

where T_{traj} tokenizes the trajectory object, Ψ_ℓ are graph- or sequence-equivariant blocks with softmax and tropical heads, and D_ω decodes property distributions, local-window labels, incompatibility warnings, and control actions.

The full oracle is more expensive than a head. It should usually be teacher-side. It becomes deployment-relevant only when it can be distilled into a small head, a small set of tropical support codes, a retrieval gate, or a behavior-control adapter that passes the held-out bits-per-byte gate.

4.3 Hybrid design

In practice one trains a ladder.

1. Frozen pooled head: train on existing trajectories with no base-model updates.
2. Low-rank adapter head: allow a small projection of hidden states to specialize.
3. Local-window oracle: add labels for short windows and tool/evidence events.
4. Full oracle transformer: train as a teacher for hard labels.
5. Distilled deployment head: export only useful property logits, gates, or low-rank controls.

This ladder avoids a common failure: building a large judge model before proving that trajectory labels improve the base model.

5 Tropical property fans and margin geometry

5.1 Property fans

Let $s \in \mathbb{R}^d$ be a trajectory summary. For property p , the tropical logit $g_p(s) = \max_r \{b_{pr} + \langle a_{pr}, s \rangle\}$ has a Newton polytope

$$P_p = \text{Conv}\{(a_{pr}, b_{pr}) : 1 \leq r \leq R_p\} \subset \mathbb{R}^{d+1}. \quad (5.1)$$

The active branch is the exposed face of this lifted polytope under $(s, 1)$. The collection of regions with fixed active branch is a normal-fan-like polyhedral complex.

Definition 5.1 (Property fan). The property fan $\Sigma_{\mathcal{P}}$ of an oracle head is the common refinement of the active-branch decompositions of all property logits g_p , all pairwise class differences $g_p - g_q$, and all incompatibility guard functions. A chamber $\sigma \in \Sigma_{\mathcal{P}}$ is a region of trajectory-summary space on which every tropical branch and every class ordering is fixed.

Proposition 5.2 (Polyhedrality of property chambers). *Every chamber of $\Sigma_{\mathcal{P}}$ is an intersection of finitely many affine half-spaces. In particular, it is a polyhedron.*

Proof. For a fixed branch r of property p to be active, one requires

$$b_{pr} + \langle a_{pr}, s \rangle \geq b_{pu} + \langle a_{pu}, s \rangle \quad (5.2)$$

for every competing branch u . Each condition is an affine half-space. Class orderings and guard inequalities are also finite affine comparisons once active branches are fixed. A chamber is the intersection of all such finite conditions, hence a polyhedron. $\square$

Definition 5.3 (Trajectory-to-wall margin). For a chamber σ containing s , the property margin is

$$\Delta_{\mathcal{P}}(s) = \min_{H \in \mathcal{W}(\sigma)} \frac{\text{dist}_{\text{signed}}(s, H)}{1 + \|n_H\|_2}, \quad (5.3)$$

where $\mathcal{W}(\sigma)$ is the set of chamber walls and n_H is the wall normal. The unnormalized tropical margin drops the denominator and uses the affine gap.

Large margins mean the classifier is stable under small hidden-state perturbations, projection error, quantization, and minor context changes. Small margins identify trajectories requiring escalation, more evidence, or full oracle processing.

5.2 Tropical decision stability

Theorem 5.4 (Tropical active-branch stability). *Let $g(s) = \max_r \{b_r + \langle a_r, s \rangle\}$. Suppose branch r^* is uniquely active at s , and define the margin*

$$\Delta(s) = \min_{u \neq r^*} [(b_{r^*} - b_u) + \langle a_{r^*} - a_u, s \rangle]. \quad (5.4)$$

If $\|s - \tilde{s}\|_2 \leq \eta$ and $\eta \max_u \|a_{r^} - a_u\|_2 < \Delta(s)/2$, then r^* remains uniquely active at $\tilde{s}$.*

Proof. For any competitor u , the active gap at $\tilde{s}$ is

$$(b_{r^*} - b_u) + \langle a_{r^*} - a_u, \tilde{s} \rangle = (b_{r^*} - b_u) + \langle a_{r^*} - a_u, s \rangle + \langle a_{r^*} - a_u, \tilde{s} - s \rangle \quad (5.5)$$

$$\geq \Delta(s) - \|a_{r^*} - a_u\|_2 \|\tilde{s} - s\|_2 \quad (5.6)$$

$$> \Delta(s) - \Delta(s)/2 > 0. \quad (5.7)$$

Thus every competitor remains strictly worse than r^* , so r^* remains uniquely active. $\square$

Corollary 5.5 (Quantized oracle safety). *If a quantizer Q satisfies $\|Q(s) - s\|_2 \leq \eta$ and the margin condition of the theorem holds, then the quantized oracle has the same active branch and the same binary decision for every threshold whose distance from the logit is greater than the induced logit perturbation.*

This result justifies storing compact oracle codes. If an exported head is mostly used in high-margin regimes, int8 or product-quantized summaries can preserve decisions. Low-margin cases should be escalated, not silently compressed.

6 Structured oracle losses

6.1 Proper scoring and calibration

The basic multi-label oracle loss is

$$\mathcal{L}_{\text{CE}} = - \sum_{p \in \mathcal{P}} [y_p \log \hat{y}_p + (1 - y_p) \log(1 - \hat{y}_p)]. \quad (6.1)$$

For ordinal labels, use cumulative-link or threshold losses. For continuous labels, use Gaussian, beta, or quantile losses depending on the scale. Cross-entropy alone is not enough; the oracle must be calibrated because thresholds for safety, refusal, or escalation depend on probabilities.

Theorem 6.1 (Binary cross-entropy is strictly proper). *Let $Y \in \{0, 1\}$ with $\mathbb{P}(Y = 1 \mid X) = p(X)$. Among predictions $q(X) \in (0, 1)$, the conditional expected binary cross-entropy*

$$\ell(q; p) = -p \log q - (1 - p) \log(1 - q) \quad (6.2)$$

is uniquely minimized at $q = p$.

Proof. Differentiate with respect to q :

$$\frac{\partial \ell}{\partial q} = -\frac{p}{q} + \frac{1 - p}{1 - q}. \quad (6.3)$$

Setting this to zero gives $p(1 - q) = (1 - p)q$, hence $q = p$. The second derivative is $p/q^2 + (1 - p)/(1 - q)^2 > 0$, so the minimizer is unique. $\square$

6.2 Poset isotonicity

If $p \Rightarrow q$ in the property poset, then $\hat{y}_p \leq \hat{y}_q$ or $\hat{y}_q \leq \hat{y}_p$ depending on convention. We fix the convention $p \preceq q$ means q implies p , so $\hat{y}_p \geq \hat{y}_q$. The isotonic penalty is

$$\mathcal{L}_{\text{iso}} = \sum_{p \preceq q} [\hat{y}_q - \hat{y}_p]_+^2. \quad (6.4)$$

Proposition 6.2 (Projection to the isotonic cone). *Let*

$$\mathcal{C}_{\text{iso}} = \{z \in \mathbb{R}^m : z_p \geq z_q \text{ whenever } p \preceq q\}. \quad (6.5)$$

The Euclidean projection $\Pi_{\mathcal{C}_{\text{iso}}}(u)$ exists, is unique, and is nonexpansive.

Proof. The set $\mathcal{C}_{\text{iso}}$ is a closed convex cone because it is an intersection of finitely many closed half-spaces $z_p - z_q \geq 0$. Projection onto a nonempty closed convex subset of a Hilbert space exists and is unique. The projection map onto a closed convex set is firmly nonexpansive, hence nonexpansive. $\square$

During inference one may project raw logits or probabilities to $\mathcal{C}_{\text{iso}}$ before making decisions. During training, the differentiable penalty encourages the same structure without solving a projection at every step.

6.3 Incompatibility penalties

Let $\mathcal{F}_{\min}$ be the minimal nonfaces of $\Delta_{\mathcal{P}}$. The Stanley-Reisner-style incompatibility penalty is

$$\mathcal{L}_{\text{inc}} = \sum_{F \in \mathcal{F}_{\min}} \prod_{p \in F} \hat{y}_p. \quad (6.6)$$

This penalty is high only when all properties in a forbidden set are simultaneously high. A log-sum-exp softened variant is

$$\mathcal{L}_{\text{inc}}^{\tau} = \sum_{F \in \mathcal{F}_{\min}} \tau \log \left(1 + \exp \left(\frac{\sum_{p \in F} \log(\hat{y}_p + \varepsilon) - \kappa_F}{\tau} \right) \right). \quad (6.7)$$

Lemma 6.3 (Minimal nonface gradients concentrate on joint violations). *For $F \in \mathcal{F}_{\min}$, the gradient of $\prod_{p \in F} \hat{y}_p$ with respect to $\hat{y}_q$ is $\prod_{p \in F \setminus \{q\}} \hat{y}_p$ if $q \in F$, and zero otherwise.*

Proof. This is the elementary product rule. Variables outside F do not appear. For $q \in F$, differentiating the product removes the factor $\hat{y}_q$ and leaves all other factors. $\square$

Thus the penalty does not waste gradient on non-violating labels. It acts most strongly when the rest of a forbidden combination is already high.

6.4 Tropical margin and active-support losses

For a property branch margin $\Delta_p(\tau)$, define

$$\mathcal{L}_{\text{margin}} = \sum_{p \in \mathcal{P}} w_p [\gamma_p - \Delta_p(\tau)]_+^2. \quad (6.8)$$

For a label with teacher support A_p^* , define an active-support loss

$$\mathcal{L}_{\text{support}} = \sum_p \text{CE}(\text{onehot}(A_p^*), \text{softmax}(\ell_{p1}, \dots, \ell_{pR_p})), \quad (6.9)$$

where $\ell_{pr} = b_{pr} + \langle a_{pr}, s \rangle$. This teaches not only what label to assign but why the label was assigned.

6.5 Sheaf gluing over trajectory windows

A trajectory can be covered by windows $U_i = [t_i, t_i + \ell]$. The oracle produces local labels $s_i \in \mathcal{F}(U_i)$. Overlaps should agree unless a property genuinely changes.

Definition 6.4 (Trajectory label presheaf). Let $\mathcal{U}$ be a cover of a trajectory index set $\{0, \dots, T\}$. A label presheaf $\mathcal{F}$ assigns to each window $U \in \mathcal{U}$ a vector space or probability simplex $\mathcal{F}(U)$ of local property judgments, and to each inclusion $V \subset U$ a restriction map $\rho_{UV} : \mathcal{F}(U) \rightarrow \mathcal{F}(V)$.

The gluing loss is

$$\mathcal{L}_{\text{glue}} = \sum_{i < j} \|\rho_{U_i, U_i \cap U_j} s_i - \rho_{U_j, U_i \cap U_j} s_j\|_2^2. \quad (6.10)$$

Theorem 6.5 (Finite sheaf gluing criterion). *Assume $\mathcal{F}$ is a sheaf of vector spaces on a finite trajectory cover and $\{s_i \in \mathcal{F}(U_i)\}$ satisfies $\rho_{U_i, U_i \cap U_j} s_i = \rho_{U_j, U_i \cap U_j} s_j$ for all overlaps. Then there is a unique global section $s \in \mathcal{F}(\cup_i U_i)$ restricting to every s_i .*

Proof. This is the sheaf gluing axiom specialized to a finite cover. Existence follows because pairwise-compatible local sections glue to a section on the union; uniqueness follows from the locality axiom: if two global sections have identical restrictions to every element of the cover, they are equal. $\square$

For neural training, the loss does not assert exact sheaf behavior. It supplies a local-to-global regularizer: a trajectory should not be labeled **grounded** on one window and **unsupported** on an overlapping window without an explicit event explaining the transition.

6.6 Composite objective

The default oracle training objective is

$$\begin{aligned} \mathcal{L}_{\text{oracle}} = & \mathcal{L}_{\text{CE}} + \lambda_{\text{iso}}\mathcal{L}_{\text{iso}} + \lambda_{\text{inc}}\mathcal{L}_{\text{inc}} + \lambda_{\text{margin}}\mathcal{L}_{\text{margin}} + \lambda_{\text{support}}\mathcal{L}_{\text{support}} \\ & + \lambda_{\text{glue}}\mathcal{L}_{\text{glue}} + \lambda_{\text{cal}}\mathcal{L}_{\text{cal}} + \lambda_{\text{byte}}\mathcal{L}_{\text{byte}} + \lambda_{\text{distill}}\mathcal{L}_{\text{distill}}. \end{aligned} \quad (6.11)$$

The byte term penalizes large exported codes and long oracle rationales. The calibration term can be expected calibration error, Brier score, temperature-scaled validation NLL, or slice calibration over property families.

7 Oracle-guided training and control

7.1 From labels to training actions

The oracle produces actions, not merely labels. Common actions are:

1. discard or downweight low-quality training trajectories;
2. construct preference pairs from high-quality and low-quality completions;
3. shape GFlowNet or search rewards;
4. gate tool calls or require additional verification;
5. restrict retrieval to fresh, grounded, permission-compatible memory;
6. adjust decoding style through low-rank behavior vectors;
7. stop context packing when marginal utility is negative;
8. escalate low-margin or high-risk cases to full oracle or human review.

7.2 Side information and bits-per-byte

A central question is why oracle labels can improve compression. If the oracle output $Z = \mathcal{O}(\tau)$ is predictive of future bytes X , then conditioning on Z can lower log loss. The following standard information-theoretic identity is the cleanest statement.

Theorem 7.1 (Side-information log-loss improvement). *Let X be held-out text and Z an oracle signal available before predicting X . The optimal expected log-loss improvement from conditioning on Z , measured in bits, is*

$$H_2(X) - H_2(X | Z) = I_2(X; Z), \quad (7.1)$$

where H_2 and I_2 use base-2 logarithms. Per byte, the maximum expected improvement is $I_2(X; Z)/\mathbb{E}|X|_{\text{bytes}}$ when lengths are deterministic or after the corresponding length-normalized conditioning.

Proof. The optimal code without side information uses $p(x)$ and has expected length $H_2(X)$. The optimal code with side information uses $p(x | z)$ and has expected length $H_2(X | Z)$. Their difference is the mutual information identity $I_2(X; Z) = H_2(X) - H_2(X | Z)$. $\square$

The theorem is not a guarantee that a learned oracle improves bits-per-byte. It says the available improvement is exactly the predictive information that survives training, quantization, and deployment costs. This is why oracle labels should be selected for downstream predictivity, not subjective appeal.

7.3 Bayes thresholds for risk properties

Proposition 7.2 (Cost-sensitive threshold). *Let $p = \mathbb{P}(Y = 1 | \tau)$ be a calibrated risk probability. If blocking a safe trajectory has cost C_{FP} and allowing a risky trajectory has cost C_{FN} , the Bayes action is to block when*

$$p > \frac{C_{\text{FP}}}{C_{\text{FP}} + C_{\text{FN}}}. \quad (7.2)$$

Proof. Allowing has expected cost pC_{FN} . Blocking has expected cost $(1 - p)C_{\text{FP}}$. Block when $(1 - p)C_{\text{FP}} < pC_{\text{FN}}$, which rearranges to the stated inequality. $\square$

This proposition is why calibration is not cosmetic. A miscalibrated oracle produces wrong risk thresholds even if its ranking performance is high.

7.4 Min-plus behavior control

Some control decisions are sequential. For example, a long-context answer may pass through states `retrieve`, `tool-call`, `verify`, `summarize`, `answer`, with costs for tone drift, risk, latency, and context bytes. This is a shortest-path problem in a control graph.

Definition 7.3 (Behavior-control graph). A behavior-control graph is a directed graph $\mathcal{B} = (Q, E)$ whose vertices are control states and whose edges $q \rightarrow q'$ have costs

$$c_t(q, q') = c_{\text{task}} + c_{\text{risk}} + c_{\text{tone}} + c_{\text{byte}} + c_{\text{latency}} - c_{\text{evidence}}. \quad (7.3)$$

The best control path for a trajectory prefix is the min-plus dynamic program

$$D_{t+1}(q') = \min_{q: q \rightarrow q'} \{D_t(q) + c_t(q, q')\}. \quad (7.4)$$

Theorem 7.4 (Tropical oracle implements finite-horizon control). *For a finite behavior-control graph with bounded costs, a min-plus tropical attention block with edge masks can compute one Bellman relaxation exactly, and a stack of T such blocks computes the finite-horizon optimal control values.*

Proof. For each target state q' , mask keys to predecessor states q with edges $q \rightarrow q'$. Store candidate values $D_t(q) + c_t(q, q')$. A min-plus head computes their minimum, which is exactly the Bellman update. Repeating the block T times computes the dynamic program by induction on horizon. $\square$

7.5 Oracle-guided policy improvement

Let π be a trajectory-generating policy and let $Q^{\mathcal{O}}(\tau, a)$ be an oracle-estimated action value for a training action or reasoning move. Use it for reward shaping only if it is calibrated and if off-policy evaluation passes.

Proposition 7.5 (Conservative oracle improvement). *Assume a finite action space and a calibrated lower confidence bound $\underline{Q}(\tau, a) \leq Q^*(\tau, a)$ with probability at least $1 - \delta$ uniformly over candidate actions. If a new policy π' is obtained by choosing only actions whose lower confidence bound exceeds the baseline action value by at least $\eta > 0$, then with probability at least $1 - \delta$, every changed decision improves true value by at least η .*

Proof. On the event that all lower bounds are valid, $\underline{Q}(\tau, a) \leq Q^*(\tau, a)$. The decision rule changes from baseline action a_0 to a only when $\underline{Q}(\tau, a) \geq \underline{Q}^*(\tau, a_0) + \eta$ or when a lower bound for the baseline is included with the corresponding conservative comparison. Therefore $Q^*(\tau, a) \geq \underline{Q}(\tau, a) \geq Q^*(\tau, a_0) + \eta$. The probability statement follows from the assumed simultaneous confidence event. $\square$

This is deliberately conservative. It prevents a beautiful but unreliable oracle from steering the base model aggressively.

8 Label acquisition and weak supervision

8.1 Label sources

Trajectory labels can come from several sources, each with different reliability.

1. Exact task checkers: unit tests, math answer checkers, theorem provers, graph algorithms, symbolic calculators.
2. Verifiers: citation checkers, proof checkers, tool-call schema checkers, retrieval-grounding checkers.
3. Human rubrics: tone, helpfulness, refusal appropriateness, uncertainty expression, and personality corridor.
4. Synthetic perturbations: inject stale memories, remove citations, corrupt tool outputs, add irrelevant context, alter tone, or create contradiction pairs.
5. Cross-model judges: teacher models label difficult qualitative properties, followed by calibration and bias audits.
6. Self-consistency and multiplicity: repeated tropical or graph-of-thought samples provide support counts and disagreement measures.

8.2 Weak-label model

Let $\lambda_j(\tau) \in \{-1, 0, 1\}$ be weak labeling functions for a binary property, where 0 means abstain. A simple latent model is

$$\mathbb{P}(\lambda_j = y \mid Y = y) = \alpha_j, \quad \mathbb{P}(\lambda_j = -y \mid Y = y) = \beta_j, \quad (8.1)$$

with abstention probability $1 - \alpha_j - \beta_j$. Estimate α_j, β_j on a small human-labeled calibration set or by agreement modeling with identifiability assumptions. Use posterior labels as soft targets.

8.3 Synthetic perturbation examples

Example 8.1 (Grounding perturbation). Start with a grounded answer trajectory whose citations support every factual claim. Replace one cited memory node with an unrelated but semantically similar node. The answer text may remain fluent, but the citation-support label should flip from high to low. The tropical oracle should identify the replaced node as the active support for the grounding-risk logit.

Example 8.2 (Tone perturbation). Take a correct concise answer and add unnecessary flattery, hedging, or excessive verbosity. Task accuracy stays fixed, while tone and efficiency labels change. This separates epistemic quality from style control.

Example 8.3 (Tool-discipline perturbation). Give two trajectories with the same final answer: one uses a necessary calculation tool, the other calls tools repeatedly without new information. Correctness is identical but tool discipline, latency, and context-token utility differ.

Synthetic perturbations are not a substitute for real labels. They are useful because they create contrastive pairs where the target property changes while many confounders remain fixed.

9 Training paradigms

9.1 Default staged recipe

1. Generate trajectories from several base checkpoints across math, coding, retrieval, tool-use, memory, long-context, and safety-style tasks.
2. Store every trajectory as a typed packet sequence with event ids, graph links, active tropical supports, verifier metrics, and context references.
3. Define an initial property taxonomy of 12–20 labels. Do not start with 100 labels.
4. Label a calibration subset with exact checkers, human rubrics, and weak supervision.
5. Train a frozen compact oracle head. Report AUROC, F1, calibration, slice errors, and active-support explanations.
6. Use the head only for data filtering or trajectory rejection. Train a base-model ablation.
7. If verified quality improves without bits-per-byte regression, add preference pairs and reward shaping.
8. Train a full oracle transformer only for labels that require local evidence spans, tool schemas, graph neighborhoods, or long-context ordering.
9. Distill useful oracle signals into a compact head, retrieval gate, or low-rank behavior adapter.
10. Export only components with positive held-out utility after artifact-byte accounting.

9.2 Pseudocode: oracle head training

```

Input: base model Phi_theta, trajectory dataset D, property labels Y,
       property poset P, incompatibility nonfaces F_min
Freeze Phi_theta initially.
for epoch in 1..E:
    for batch trajectories tau in D:
        s = pooled_summary(Phi_theta, tau)
        logits = tropical_oracle_head(s)          # max over affine branches
        probs = calibrated_sigmoid(logits)
        L = CE(probs, Y)
        L += lambda_iso * poset_isotonic_penalty(probs, P)
        L += lambda_inc * incompatibility_penalty(probs, F_min)
        L += lambda_margin * tropical_margin_penalty(logits)
        L += lambda_glue * local_window_gluing_penalty(tau, probs)
    update oracle parameters

```

validate calibration, slice error, active support explanations promote to filtering only if downstream ablation improves verified quality
--

9.3 Pseudocode: oracle-assisted base-model training

Input: base training data, trained oracle $\mathcal{O}$, conservative thresholds for each prompt x : sample K candidate trajectories $\tau_1, \dots, \tau_K$ from base model score each τ_k with oracle $\mathcal{O}$ discard candidates with high calibrated risk or bad grounding choose preference pairs $(\tau_{\text{good}}, \tau_{\text{bad}})$ when oracle margin is large compute supervised / preference / reward-shaping loss update base model periodically run held-out bits-per-byte and verifier evaluation if bits-per-byte worsens beyond tolerance, roll back oracle-assisted update
--

9.4 Pseudocode: test-time behavior projection

Input: partial trajectory $\tau_{\{ \leq t \}}$, oracle $\mathcal{O}$, threshold policy, context budget scores = $\mathcal{O}(\tau_{\{ \leq t \}})$ if scores.risk_high and scores.margin_high: require refusal or additional verification elif scores.grounding_low: retrieve more evidence or cite uncertainty elif scores.context_utility_negative: stop context packing elif scores.tone_drift_high: apply low-rank style correction else: continue generation log oracle action and include it in held-out audit

10 Applications to reasoning quality and behavior control

10.1 Accuracy and proof rigor

An oracle can detect likely proof gaps before the final answer. For mathematical reasoning, label local windows by whether each claim follows from previous statements, retrieved facts, or tool outputs. A tropical logit is suitable because one severe proof gap can dominate the trajectory label. The active support identifies the precise window responsible.

10.2 Grounding and citation support

Grounding labels compare answer spans to retrieved evidence nodes. The context graph from TropicalGT III supplies provenance and adjacency. A local label g_i says whether span i is supported. The global grounding score may be min-plus over spans:

$$G_{\text{grounded}} = \min_i g_i, \quad (10.1)$$

because a single unsupported factual claim can make the answer unreliable. Equivalently, max-plus can model risk as $G_{\text{risk}} = \max_i (1 - g_i)$.

10.3 Compliance and refusal appropriateness

Compliance is not always good. A refusal can be correct or incorrect depending on user intent and policy. The oracle should therefore label both **helpful-compliance** and **refusal-appropriateness**, with an incompatibility complex preventing incoherent simultaneous predictions. The Bayes threshold for escalation should use calibrated risk probabilities and cost-sensitive thresholds.

10.4 Tone and personality corridor

Tone and personality are secondary to truth and safety, but still operational. A model can be accurate yet overly effusive, evasive, sycophantic, terse, or miscalibrated. Define a desired style vector c^* and observed trajectory style vector $c(\tau)$. A tropical corridor loss emphasizes the worst deviation:

$$\mathcal{L}_{\text{tone}} = \max_j \alpha_j |c_j(\tau) - c_j^*|. \quad (10.2)$$

This is preferable to an average loss when one severe style defect dominates user experience.

10.5 Efficiency and context-token utility

Let a trajectory use context blocks $B_1, \dots, B_k$ with byte costs b_i and oracle-estimated marginal utilities u_i . The long-context policy should include blocks in decreasing tropical utility until

$$u_i - \lambda b_i < 0. \quad (10.3)$$

The oracle can label **context-waste** when many low-utility blocks enter the context. This directly supports held-out bits-per-byte: fewer irrelevant tokens should lower confusion and improve compression when the oracle is correct.

10.6 Tool and MCP discipline

For tool calls, labels include schema validity, permission safety, argument relevance, result usage, redundancy, and error handling. A tool call that is syntactically valid but unused is inefficient. A tool result that contradicts the answer is a grounding failure. A tool call with insufficient permission is a risk even if it would answer the task. The oracle can gate calls before execution, require additional user confirmation, or route to a safer tool.

11 Evaluation metrics and ablation design

11.1 Oracle metrics

Report standard metrics: AUROC, AUPRC, macro/micro F1, Brier score, expected calibration error, temperature-scaled NLL, per-slice calibration, and active-support explanation accuracy. For ordinal labels, report quadratic weighted kappa and ordinal calibration. For incompatibility, report forbidden-pair violation rate.

11.2 Downstream model metrics

More important are downstream metrics:

- held-out bits-per-byte;
- task accuracy and verifier agreement;

- hallucination and unsupported-claim rate;
- tool-call success and tool-call waste;
- retrieval precision, recall, and graph-closure quality;
- context-token utility and latency;
- refusal appropriateness and unsafe-compliance rate;
- tone corridor and personality stability;
- answer readiness and stopping-time efficiency;
- artifact bytes and exported parameter count.

11.3 Ablation ladder

A rigorous evaluation should include at least:

1. base model without oracle;
2. oracle head trained but not used;
3. oracle filtering only;
4. oracle SFT weighting;
5. oracle preference training;
6. oracle GFlowNet reward shaping;
7. oracle retrieval/memory gating;
8. oracle tone/style steering;
9. full oracle transformer teacher;
10. distilled compact deployment head.

Each step must report downstream metrics and confidence intervals. A component that improves oracle F1 but not downstream model behavior should remain diagnostic.

12 Implementation details

12.1 Trajectory record schema

A JSONL trajectory record should contain:

```
{
  "task_id": "...",
  "prompt_hash": "...",
  "trajectory_id": "...",
  "packets": [
    {"t": 0,
     "visible_span_ids": [...],
     "graph_nodes": [...], "graph_edges": [...],
     "retrieved_context_ids": [...],
     "tool_events": [...],
     "active_supports": [...],
     "tropical_margins": [...],
```

```

    "verifier_scores": {...},
    "behavior_features": {...}
  ],
  "final_output_id": "...",
  "labels": {"correct": 1, "grounded": 0.7, "tone_match": 1, ...},
  "label_sources": {...},
  "byte_accounting": {"prompt_bytes": 0, "context_bytes": 0, "artifact_bytes": 0}
}

```

Store raw private data separately from trainable summaries when privacy matters. The oracle should be able to run on hashed ids, compressed evidence summaries, and verifier outputs when raw text cannot be retained.

12.2 Feature extraction

Useful features include:

- final hidden pool and per-window hidden pools;
- active tropical support ids and margins;
- retrieval graph statistics and citation coverage;
- tool-call schema validity and result usage;
- verifier scores and contradiction flags;
- persistence/barcode summaries for graph-of-thought structure;
- tone/personality embeddings and verbosity statistics;
- context-token marginal utility estimates;
- uncertainty and calibration features.

12.3 Recommended hyperparameters

Initial runs should use a small taxonomy: 12–20 labels, tropical head rank 4–16 per label, hidden summary dimension 256–1024, local windows of 4–16 packets, margin target $\gamma \in [0.25, 1.0]$, incompatibility weight 10^{-3} to 10^{-2} , isotonic weight 10^{-3} to 10^{-2} , and calibration every epoch on a held-out labeled set. Train the oracle head before updating the base model. Only after downstream improvements should one train a full oracle transformer.

12.4 Reproducibility protocol

Report random seeds, model checkpoints, exact property taxonomy, label instructions, weak label functions, calibration set size, source manifests, tool schemas, context graph construction rules, prompt/task split rules, privacy filters, and artifact-byte accounting. For every claimed improvement, report matched ablations with paired bootstrap intervals or randomization tests.

13 Theoretical guarantees for deployment gates

13.1 Equivariance

Theorem 13.1 (Oracle equivariance). *Suppose trajectory tokenization is equivariant under graph relabeling, attention masks are relabeling-equivariant, all oracle projections are shared over token types,*

and property decoders use permutation-invariant pooling or graph-level tokens. Then the oracle’s graph-level property distribution is invariant under graph relabeling, and its node/window-local labels are equivariant.

Proof. Equivariant tokenization maps a relabeled trajectory to a row-permuted token table. Shared softmax and tropical attention commute with row permutations because score matrices are conjugated by the permutation matrix and row-wise max or softmax is invariant under simultaneous key reindexing. Shared feed-forward layers and residuals are row-wise, hence equivariant. A composition of equivariant maps is equivariant. Permutation-invariant pooling gives invariant graph-level labels; local decoders give permuted local labels. $\square$

13.2 Blockwise long-context oracle aggregation

Theorem 13.2 (Exact blockwise tropical aggregation). *Let a risk logit be $g(\boldsymbol{\tau}) = \max_{j \leq L} \{u_j\}$, where u_j is a local-window risk score. Partition the windows into blocks $B_1, \dots, B_R$. If each block returns $m_r = \max_{j \in B_r} u_j$, then $\max_r m_r = g(\boldsymbol{\tau})$, independent of block order.*

Proof. The blocks form a partition, so

$$\max_j u_j = \max_r \max_{j \in B_r} u_j. \quad (13.1)$$

The maximum operation is associative and commutative, so the result is independent of block order. $\square$

This matters for million-token contexts. A risk or support label that is naturally max-plus can be computed by streaming blocks without changing the exact mathematical result.

13.3 Oracle promotion gate

Definition 13.3 (Promotion utility). For an oracle component C , define validation utility

$$\widehat{U}(C) = \Delta \widehat{\text{bits-per-byte}} + \lambda_A \Delta \widehat{A} + \lambda_R \Delta \widehat{R} + \lambda_C \Delta \widehat{C} - \rho_B \widehat{B}_C - \rho_L \widehat{L}_C, \quad (13.2)$$

where $\widehat{B}_C$ is artifact bytes and $\widehat{L}_C$ is latency or runtime-token cost.

Proposition 13.4 (Conservative promotion rule). *Let $[L_C, U_C]$ be a valid confidence interval for $U(C)$. Promoting only when $L_C > 0$ controls the probability of promoting a non-positive-utility component by the interval failure probability.*

Proof. If the interval is valid with coverage $1 - \alpha$, then with probability at least $1 - \alpha$, $U(C) \in [L_C, U_C]$. On this event, the condition $L_C > 0$ implies $U(C) > 0$. Therefore a false promotion can occur only when the confidence interval fails, with probability at most α . $\square$

14 Objective catalogue

The following objectives are intended as a practical menu. They should not all be turned on at once. Start with audit-only metrics, test correlation with validation outcomes, then promote only the few that improve the base model.

Objective 14.1 (1. Tropical correctness margin). **Definition.** Correct trajectories should have a large tropical margin between the correct-class logit and the strongest incorrect-class logit.

$$\mathcal{L} = \mathbb{E}[\gamma - (g_{\text{correct}} - g_{\text{incorrect}}^{(1)})]_+^2 \quad (14.1)$$

Training use. Use on exact-answer math, coding, graph-algorithm, and tool tasks. Large margins support quantized deployment because decisions survive perturbations. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.2 (2. Verifier agreement classifier). **Definition.** Train the oracle to predict whether independent verifiers accept the trajectory before the final answer is known.

$$\mathcal{L} = \text{CE}(\hat{y}_{\text{verify}}, y_{\text{verify}}) + \lambda \mathcal{L}_{\text{support}} \quad (14.2)$$

Training use. Use as early stopping and trajectory rejection. Active supports should point to the proof step or tool result driving rejection. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.3 (3. Grounding worst-span risk). **Definition.** A single unsupported factual span can dominate answer risk, so the score uses max-plus over local unsupported-span logits.

$$g_{\text{ungrounded}} = \max_i \{b_i + \ell_i^{\text{unsupported}}\} \quad (14.3)$$

Training use. Use for citation-heavy answers and markdown/KG memory. Retrieve additional evidence when the risk branch has low margin or high severity. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.4 (4. Citation closure regularizer). **Definition.** A claim node, citation node, source node, and provenance node should be retrieved as a graph-closed neighborhood.

$$\mathcal{L} = \sum_{v \in R} \text{dist}(\text{closure}(v), R)^2 \quad (14.4)$$

Training use. Use with graph-structured vector databases. It improves reasoning by preventing isolated snippets from being treated as grounded evidence. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.5 (5. Tool discipline loss). **Definition.** Classify whether tool calls are necessary, schema-valid, permission-compatible, and used in the final answer.

$$\mathcal{L} = \text{CE}(\hat{y}_{\text{tool}}, y_{\text{tool}}) + \lambda \max(0, n_{\text{redundant}} - b)^2 \quad (14.5)$$

Training use. Use for MCP tool traces. Penalizes wasteful tool use without discouraging necessary computation. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.6 (6. Refusal appropriateness incompatibility). **Definition.** Safe refusal and unsafe compliance are incompatible, but refusal can also be inappropriate for benign requests.

$$\mathcal{L}_{\text{inc}} = \hat{y}_{\text{safe_refusal}} \hat{y}_{\text{unsafe_compliance}} + \hat{y}_{\text{benign}} \hat{y}_{\text{overrefusal}} \quad (14.6)$$

Training use. Use calibrated thresholds; do not optimize refusal style independently of task and safety labels. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.7 (7. Tone corridor max deviation). **Definition.** Aggregate tone/personality deviations by the worst coordinate rather than the average.

$$\mathcal{L}_{\text{tone}} = \max_j \alpha_j |c_j(\boldsymbol{\tau}) - c_j^*| \quad (14.7)$$

Training use. Useful when one severe style defect dominates the interaction. Keep secondary to truth and safety. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.8 (8. Personality stability under evidence updates). **Definition.** The agent’s behavioral profile should not drift merely because more evidence was retrieved, unless the task requires a style change.

$$\mathcal{L} = \sum_t \|c_{t+1} - c_t\|^2 \mathbf{1}\{\Delta E_t \text{ epistemic only}\} \quad (14.8)$$

Training use. Use for long-context trajectories where retrieval may accidentally alter style. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.9 (9. Uncertainty calibration loss). **Definition.** Calibrate confidence to correctness and verifier agreement across property slices.

$$\mathcal{L} = \text{Brier}(\hat{p}_{\text{correct}}, y) + \lambda \text{ECE} \quad (14.9)$$

Training use. Useful for deciding whether to answer, retrieve more, or escalate. Miscalibration invalidates thresholds. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.10 (10. Context-token utility classifier). **Definition.** Predict whether each context block improves answer quality enough to justify its byte cost.

$$u_i = \Delta \hat{A}_i + \lambda \Delta \hat{R}_i - \rho b_i \quad (14.10)$$

Training use. Use for long-context packing. Stop inserting blocks when marginal utility becomes negative. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.11 (11. Memory freshness risk). **Definition.** Classify whether a retrieved memory is stale relative to the task’s temporal requirements.

$$g_{\text{stale}} = \max_i \{ \ell_i^{\text{relevance}} + \ell_i^{\text{age}} - \ell_i^{\text{validity}} \} \quad (14.11)$$

Training use. Use with markdown memories and MCP resources. Forces retrieval to account for time validity, not just semantic similarity. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.12 (12. Contradiction cycle detector). **Definition.** A cycle in the evidence graph can be productive or contradictory. Classify cycles by relation labels and verifier outcomes.

$$\mathcal{L} = \text{CE}(\hat{y}_{\text{cycle}}, y_{\text{cycle}}) + \lambda \|\partial c\|^2 \quad (14.12)$$

Training use. Connects TropicalGT III retrieval barcodes to oracle labels. Useful for multi-source synthesis. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.13 (13. Patchworked answer gluing). **Definition.** Local answer sections should agree on overlaps: notation, assumptions, citations, and tone.

$$\mathcal{L} = \sum_{i < j} \|\rho_i s_i - \rho_j s_j\|^2 \quad (14.13)$$

Training use. Use on long reports and multi-step reasoning. Prevents local chunks from composing into a globally inconsistent answer. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.14 (14. Answer-readiness stopping loss). **Definition.** Classify whether additional reasoning/context is expected to improve the answer.

$$\mathcal{L} = \text{CE}(\hat{y}_{\text{ready}}, y_{\text{ready}}) + \lambda \max(0, T - T^*)^2 \quad (14.14)$$

Training use. Use as a stopping-time controller. Reduces overthinking and context waste. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.15 (15. Active-support faithfulness). **Definition.** The oracle’s branch explanation should point to the trajectory window responsible for the label.

$$\mathcal{L} = \text{CE}(\text{softmax}(\ell_{pr}), A_p^*) \quad (14.15)$$

Training use. Requires teacher explanations or synthetic perturbations. Useful for debugging classifier failures. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.16 (16. Low-margin escalation). **Definition.** Low property margins indicate unstable decisions and should trigger more evidence or a stronger oracle.

$$\text{escalate} = \mathbf{1}\{\Delta_{\mathcal{P}} < \gamma\} \quad (14.16)$$

Training use. This is a policy objective rather than a differentiable loss. It improves reliability under quantization and projection. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.17 (17. Bits-per-byte utility predictor). **Definition.** Predict whether an oracle action will improve held-out bits-per-byte before applying it broadly.

$$\mathcal{L} = \left(\hat{\Delta} \text{ bits-per-byte} - \Delta \text{ bits-per-byte} \right)^2 \quad (14.17)$$

Training use. Use to prevent structural objectives from becoming mathematical ornament. Actions with negative predicted utility remain teacher-side. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.18 (18. Preference-pair construction). **Definition.** Use oracle labels to construct pairs of trajectories differing in one property while holding task fixed.

$$\mathcal{L} = -\log \sigma(r(\tau^+) - r(\tau^-)) \quad (14.18)$$

Training use. Generates preference data for DPO/RL-style training. Requires high-margin labels to avoid noisy preferences. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.19 (19. Risk-sensitive min-plus control). **Definition.** Choose actions in a behavior-control graph by minimizing risk, latency, byte cost, and task loss.

$$D_{t+1}(q') = \min_q \{D_t(q) + c_t(q, q')\} \quad (14.19)$$

Training use. Use for tool-call, refusal, verification, and context-packing policies. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.20 (20. Oracle distillation loss). **Definition.** Distill a full oracle transformer into a compact head or low-rank adapter.

$$\mathcal{L} = \text{KL}(p_{\text{full}}(\cdot \mid \tau) \| p_{\text{small}}(\cdot \mid \tau)) \quad (14.20)$$

Training use. Export only if the compact student preserves downstream benefits. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.21 (21. Forbidden-property barrier). **Definition.** Use the incompatibility complex to penalize impossible or harmful label combinations.

$$\mathcal{L} = \sum_{F \in \mathcal{F}_{\min}} \prod_{p \in F} \hat{y}_p \quad (14.21)$$

Training use. This is especially useful for safety/compliance/tone combinations where independent sigmoids produce incoherence. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.22 (22. Ordinal severity calibration). **Definition.** Many labels are severity levels rather than binary events.

$$\mathbb{P}(Y \geq k) = \sigma(g - \theta_k), \quad \theta_1 < \dots < \theta_K \quad (14.22)$$

Training use. Use for hallucination severity, tone drift, verbosity mismatch, and risk level. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.23 (23. Trajectory diversity quality). **Definition.** Reward diverse candidate trajectories only when diversity improves support, not when it creates noisy duplicates.

$$\mathcal{L} = -\text{quality} + \lambda \text{duplicate_sim} \quad (14.23)$$

Training use. Use with GFlowNet or sampling. Avoid treating repeated equivalent bad trajectories as evidence. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.24 (24. Ergodic non-thrashing score). **Definition.** Repeated reasoning updates should not bounce between incompatible chambers without new evidence.

$$\mathcal{L} = \sum_t \mathbf{1}\{c_t \neq c_{t+1}\} \mathbf{1}\{\Delta E_t = 0\} \quad (14.24)$$

Training use. Derived from TropicalGT II ergodic cell occupation. Useful for stability and token efficiency. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.25 (25. Retriever-oracle agreement). **Definition.** If the oracle says the answer is ungrounded, retrieval diagnostics should show missing or weak evidence.

$$\mathcal{L} = \|\hat{y}_{\text{ungrounded}} - f(B(q), R(q))\|^2 \quad (14.25)$$

Training use. Connects TropicalGT III retrieval barcodes to trajectory labels. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.26 (26. MCP permission mask audit). **Definition.** High relevance must not override tool/resource permissions.

$$\mathcal{L} = \sum_v \hat{y}_{\text{use}}(v) \mathbf{1}\{\neg \text{allowed}(v)\} \quad (14.26)$$

Training use. Use as a hard mask at inference and an audit penalty during training. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.27 (27. Output-style compression objective). **Definition.** Classify whether the final answer has excessive redundancy relative to user need.

$$\mathcal{L} = \text{CE}(\hat{y}_{\text{verbose}}, y_{\text{verbose}}) + \lambda \text{ bytes}(y) \quad (14.27)$$

Training use. Improves bits-per-byte and user-facing clarity when balanced against completeness. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.28 (28. Proof-obligation coverage). **Definition.** Every introduced proof obligation should be discharged, cited, or explicitly marked unresolved.

$$\mathcal{L} = \sum_o \max(0, 1 - \text{covered}(o)) \quad (14.28)$$

Training use. Use in mathematical and scientific reasoning. Prevents elegant but incomplete arguments. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.29 (29. Counterfactual label consistency). **Definition.** Small irrelevant perturbations should not change labels; targeted perturbations should change only intended labels.

$$\mathcal{L} = \|O(\tau) - O(\tau_{\text{irrelevant}})\|^2 + \|M(O(\tau) - O(\tau_{\text{targeted}}))\|^2 \quad (14.29)$$

Training use. Synthetic perturbation framework for robust oracle training. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.30 (30. Artifact-aware Pareto gate). **Definition.** Select oracle components on a Pareto frontier of quality, risk, latency, artifact bytes, and bits-per-byte.

$$\max_C (\Delta Q(C), -\Delta R(C), -B_C, -L_C, \Delta \text{ bits-per-byte}(C)) \quad (14.30)$$

Training use. Avoid a single scalar hiding deployment tradeoffs. Report the full frontier. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.31 (31. Local rationale sufficiency). **Definition.** The oracle’s public rationale should be concise but sufficient to justify an action without exposing hidden chain-of-thought.

$$\mathcal{L} = \text{CE}(\hat{y}_{\text{suff}}, y_{\text{suff}}) + \lambda \text{bytes}(r) \quad (14.31)$$

Training use. Useful for audit logs and user-visible explanations. Use citations and verifier status rather than hidden reasoning text. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.32 (32. Behavior-corridor projection). **Definition.** Project style/tone/personality coordinates back into an allowed convex corridor after content is fixed.

$$c' = \Pi_{\mathcal{C}}(c), \quad \mathcal{L} = \|c - c'\|^2 \quad (14.32)$$

Training use. Use only after truth/safety labels are satisfied. Style correction must not alter factual content. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

Objective 14.33 (33. Oracle uncertainty routing). **Definition.** Route high-uncertainty labels to stronger verifiers or human review rather than making silent decisions.

$$\text{route} = \mathbf{1}\{H(\hat{p}) > h_0 \text{ or } \Delta < \gamma\} \quad (14.33)$$

Training use. Improves safety and calibration; costs latency, so it must be gated by severity. **Promotion rule.** Begin audit-only, measure correlation with downstream metrics, enable as a loss only after matched ablations show improved verified quality, control reliability, or held-out bits-per-byte without unacceptable artifact or latency cost.

15 Worked examples

15.1 Mathematical proof trajectory

A proof task produces a trajectory with definitions, lemma attempts, retrieved facts, symbolic checks, and a final proof. The oracle labels local windows by proof-obligation coverage, verifier agreement, and unsupported inference. Suppose a proof has five obligations $o_1, \dots, o_5$ and four are discharged. The proof-obligation coverage score is

$$c_{\text{proof}} = \frac{1}{5} \sum_{i=1}^5 \mathbf{1}\{o_i \text{ discharged}\} = 0.8. \quad (15.1)$$

The global proof-risk logit can still be high if the missing obligation is central:

$$g_{\text{risk}} = \max_i \{w_i(1 - \text{covered}(o_i))\}. \quad (15.2)$$

A central missing step with $w_i = 10$ dominates minor style defects. This is exactly the kind of extremal aggregation tropical classification is designed for.

15.2 MCP tool trajectory

A user asks for a computation requiring a database query. The trajectory calls a tool with valid schema, receives a result, and uses the result in the answer. A contrasting trajectory calls three irrelevant tools and ignores two results. Both final answers may be correct. The oracle should distinguish correctness from tool discipline. Labels include **schema-valid**, **permission-valid**, **result-used**, **redundant-tool-call**, and **latency-waste**. The training action is not necessarily to forbid tools; it is to teach efficient tool choice.

15.3 Long-context memory trajectory

A long-context task retrieves 200 markdown blocks. Only 12 are cited or semantically used. The oracle predicts high context waste and low marginal utility for later blocks. TropicalGT III’s context-packing policy can then stop early. If answer quality stays fixed and held-out bits-per-byte improves, the context-utility oracle is useful. If the model becomes less grounded, the stop policy is too aggressive.

15.4 Tone/personality control trajectory

A medical-style factual query may require calm, precise, non-overconfident language. A creative-writing query may tolerate a warmer style. The oracle should not impose a single personality vector independent of task. It should classify whether the style matches user intent and risk level. In the objective hierarchy, truth, safety, and uncertainty calibration dominate style. The behavior-corridor projection is therefore applied after factual and safety checks.

16 Full oracle transformer methodology

16.1 Packet-token construction

The full oracle transformer is not a generic judge that reads only the final answer. It reads an explicitly typed sequence or graph of packets. A packet token can represent a user instruction, a model draft span, a retrieved markdown block, a knowledge-graph path, a tool-call request, a tool-call result, a verifier event, a citation edge, an active tropical support, a persistence-barcode summary, or a context-packing decision. The model receives type embeddings, local order embeddings, graph-neighborhood encodings, provenance fields, byte costs, and masks. Raw private reasoning text can be replaced by compressed state summaries when needed.

Let the token table for trajectory τ be

$$Z_0 = T_{\text{traj}}(\tau) \in \mathbb{R}^{L \times d}. \quad (16.1)$$

A hybrid oracle block has ordinary heads for smooth semantic aggregation and tropical heads for extremal evidence:

$$Z_{\ell+1} = Z_{\ell} + \text{SoftmaxAttn}_{\ell}(Z_{\ell}) + \text{TropAttn}_{\ell}(Z_{\ell}) + \text{FFN}_{\ell}(Z_{\ell}), \quad (16.2)$$

$$\text{TropAttn}_{ic}(Z) = \max_{j \in \mathcal{M}_i} \{S_{ij}(Z) + V_{jc}(Z)\}. \quad (16.3)$$

The mask $\mathcal{M}_i$ depends on type, causality, graph adjacency, permission, and local-window boundaries. Property logits are decoded by graph-level pooling plus local-window heads:

$$\hat{y}_p = \sigma \left(d_p^{\top} \text{Pool}(Z_L) + \max_{i \in W_p} e_p^{\top} Z_{L,i} + b_p \right). \quad (16.4)$$

The first term captures global context; the tropical maximum captures the most decisive local event.

Definition 16.1 (Local witness set). For a property p , the local witness set is

$$W_p(\tau) = \arg \max_{i \in W_p} \{e_p^\top Z_{L,i} + b_{pi}\}. \quad (16.5)$$

A witness is *valid* if it points to a token type that can legitimately support the property: evidence spans for grounding, tool events for tool discipline, verifier events for proof validity, and behavior spans for tone/personality labels.

A practical oracle should report witness validity. If the **unsafe-compliance** logit is controlled by an unrelated punctuation token, the oracle may still be accurate but its explanation is not useful for control. Witness validity can be trained from synthetic perturbations and human rationales.

16.2 Windowed and hierarchical inference

For long trajectories, the full oracle should run hierarchically. First classify windows of length w . Then classify segments built from window summaries. Finally classify the entire trajectory from segment summaries and a small number of high-risk/high-support witness tokens. Formally,

$$h_i^{(1)} = O_{\text{local}}(R_i, \dots, R_{i+w-1}), \quad h_j^{(2)} = O_{\text{seg}}(h_{jw}^{(1)}, \dots, h_{(j+1)w-1}^{(1)}), \quad (16.6)$$

with global scores

$$g_p(\tau) = \max_j \{a_p^\top h_j^{(2)} + b_p\}. \quad (16.7)$$

This preserves the max-plus semantics of decisive events while keeping computation manageable. It also creates a natural audit trail: global label p points to a segment, the segment points to a window, and the window points to a token.

Proposition 16.2 (Hierarchical max consistency). *If the global property risk is the maximum of local risks, and segment risks are exact maxima of window risks, then hierarchical inference returns the same risk value as flat inference over all windows.*

Proof. Let local risks be r_i . Segment j returns $m_j = \max_{i \in B_j} r_i$. The global value is $\max_j m_j = \max_j \max_{i \in B_j} r_i = \max_i r_i$, because the blocks B_j partition the windows. $\square$

16.3 Universal approximation on bounded trajectory families

The oracle need not solve all possible agent behavior in one theorem. The useful formal claim is compact-domain approximation.

Theorem 16.3 (Bounded trajectory classifier approximation). *Fix maximum packet count $T_{\max}$, maximum token count per packet, finite token types, bounded continuous attributes, and a finite property set. Let F be a continuous graph-relabeling-invariant property map on this compact padded trajectory domain. A sufficiently wide hybrid oracle transformer with shared token maps, equivariant masks, ordinary feed-forward layers, and tropical/softmax attention heads can approximate F uniformly to arbitrary accuracy.*

Proof. The bounded padded domain is a finite union of compact Euclidean cells indexed by masks, token types, graph incidence patterns, and packet lengths. On each cell, tokenization is continuous. Standard neural-network universal approximation handles continuous maps on compact subsets after fixing a cell. Shared equivariant layers and invariant pooling approximate invariant maps by symmetrizing over token permutations or by using known transformer approximation results on compact sequence domains. Tropical heads do not reduce expressivity because they can be ignored by setting their output weights to zero; they add exact max-plus gates for extremal subfunctions. A finite partition argument combines the cellwise approximations. $\square$

This theorem is deliberately scoped. It does not say that a finite oracle generalizes to arbitrary context length or arbitrary tools. It says that, under bounded experimental regimes, the architecture is not the limiting factor; data, labels, calibration, and deployment utility are.

17 Benchmark suite and data generation

17.1 Benchmark families

A meaningful TropicalGT IV evaluation should test both classification accuracy and downstream utility. We recommend six benchmark families.

Exact reasoning tasks. Graph algorithms, dynamic programming, symbolic algebra, theorem-proving fragments, and programming tasks with unit tests. Labels: correctness, proof gap, verifier agreement, efficiency, active-support faithfulness.

Grounded retrieval tasks. Question answering over retrieved documents, markdown memory, code repositories, and knowledge graphs. Labels: citation support, unsupported span, contradiction, stale evidence, retrieval closure, context waste.

MCP/tool tasks. Sandboxed tool-call plans with schemas and permissions. Labels: tool necessity, schema validity, permission validity, result usage, redundant call, unsafe action.

Long-context tasks. Context budgets 4k, 16k, 64k, 256k, 1M, 4M, 10M. Labels: answer readiness, marginal context utility, topic drift, evidence dilution, latency waste, memory freshness.

Behavior and style tasks. Controlled variations of tone, directness, uncertainty, verbosity, and personality corridor. Labels: tone match, personality stability, overconfidence, excessive hedging, verbosity mismatch.

Safety/compliance tasks. Benign requests, ambiguous requests, unsafe requests, and requests requiring refusal or safe redirection. Labels: safe compliance, safe refusal, over-refusal, unsafe compliance, escalation needed.

17.2 Counterfactual trajectory generation

A high-value dataset contains counterfactual pairs (τ, τ') where one property changes and others are held as fixed as possible. Examples:

- replace a supporting citation by an irrelevant citation while leaving answer text fixed;
- keep the final answer correct but add redundant tool calls;
- keep the same evidence but alter uncertainty language;
- keep the same prompt and tools but remove a necessary proof step;
- keep the same retrieved documents but insert stale memory with high semantic similarity;
- keep the same safety category but change the tone from neutral to patronizing.

Counterfactuals teach the oracle to separate entangled properties. They also produce high-margin preference pairs for base-model training.

17.3 Dataset split discipline

Splits should prevent leakage through prompt templates, source documents, tools, and synthetic perturbation seeds. For retrieval tasks, split by source corpus and by question family. For tool tasks, split by tool schema family. For long-context tasks, split by context length and by distractor generator. For style tasks, split by rubric writer or by transformation generator. Report both in-distribution and out-of-distribution results; a trajectory oracle that only learns a perturbation artifact is not useful.

18 Deployment patterns

18.1 Teacher-side oracle

A teacher-side oracle runs offline or during training. It may be large, inspect raw evidence, compute persistent summaries, and use full context graphs. It can label trajectories, build preference pairs, and audit failures. It is not deployed in the inference artifact unless distilled.

18.2 Shadow oracle

A shadow oracle runs during evaluation but does not affect outputs. It estimates whether oracle actions would have helped. This is the safest intermediate step before using the oracle to gate generation.

18.3 Runtime gate

A runtime gate is a small calibrated head that can trigger limited actions: retrieve more evidence, stop context packing, require verification, block an unauthorized tool call, or apply a low-rank tone correction. It must be fast, small, and high precision for high-cost interventions.

18.4 Exported behavior adapter

An exported behavior adapter is a low-rank module trained from oracle labels to make the base model less likely to enter bad trajectory chambers. It should be evaluated against style, safety, accuracy, and held-out bits-per-byte simultaneously. If it improves tone but harms truth, it fails.

18.5 Audit log

For every oracle action, log the trajectory id, property id, probability, margin, witness token, action, and downstream outcome. This makes it possible to debug false positives, false negatives, and distribution shift. The audit log should not contain private hidden chain-of-thought text unless the experiment is explicitly local and consented.

19 Additional tropical objectives for TropicalGT IV

Objective 19.1 (Slice-calibrated tropical reliability). Calibrate oracle probabilities not only globally but on slices: task family, context length, tool schema, user-intent class, safety category, and property family. Let S be a slice and $\text{ECE}(S)$ its expected calibration error. Penalize

$$\mathcal{L}_{\text{slice}} = \sum_{S \in \mathcal{S}} w_S \text{ECE}(S). \quad (19.1)$$

This prevents an oracle from being well calibrated on average but unreliable on long-context or safety-critical subsets.

Objective 19.2 (Tropical witness sparsity). A label should usually be explained by a small number of decisive windows. Let π_{pi} be a softmax over local witness scores for property p . Penalize high entropy when a sparse witness is expected:

$$\mathcal{L}_{\text{witness}} = \sum_p \mathbf{1}\{p \in \mathcal{P}_{\text{sparse}}\} H(\pi_p). \quad (19.2)$$

Use for grounding failures, unsafe tool calls, and proof gaps. Do not use when the property is genuinely distributed, such as overall verbosity.

Objective 19.3 (Refusal-boundary patchworking). Train local windows around a refusal decision to agree on user intent, policy category, and safe alternative. The loss combines local classification with overlap agreement:

$$\mathcal{L} = \mathcal{L}_{\text{local}} + \lambda \sum_{i \sim j} \|s_i - s_j\|^2. \quad (19.3)$$

This reduces abrupt refusal boundary crossings caused by one misleading retrieved block.

Objective 19.4 (Evidence multiplicity confidence). If multiple independent evidence paths support an answer, confidence can increase; duplicated paths should not. Let $\mathcal{P}$ be support paths and $\text{indep}(P, Q)$ an independence score. Define

$$m = \sum_{P \in \mathcal{P}} w(P) - \lambda \sum_{P < Q} (1 - \text{indep}(P, Q)) w(P) w(Q). \quad (19.4)$$

Use m as an oracle feature for confidence, not as a substitute for correctness.

Objective 19.5 (Tropical chamber drift penalty). Let c_t be the property-fan chamber occupied by trajectory packet R_t . Penalize chamber changes without evidence or task-state changes:

$$\mathcal{L}_{\text{drift}} = \sum_t \mathbf{1}\{c_t \neq c_{t+1}\} \exp(-\|E_{t+1} - E_t\|). \quad (19.5)$$

This encourages stable behavior and reduces hidden thrashing.

Objective 19.6 (Oracle-action counterfactual regret). When the oracle chooses an action a , estimate the outcome of alternatives a' . Penalize positive regret:

$$\mathcal{L}_{\text{regret}} = \max(0, \hat{Q}(a') - \hat{Q}(a) + \gamma). \quad (19.6)$$

Use only with conservative confidence intervals; otherwise the oracle can learn to justify its own actions.

Objective 19.7 (Long-context dilution detector). Predict whether adding more context reduces the signal-to-noise ratio. If S_k is answer support after k blocks and N_k is distractor mass, define

$$d_k = N_k - S_k. \quad (19.7)$$

A max-plus risk $\max_k d_k$ flags dilution. Use to stop packing before irrelevant memory harms reasoning.

Objective 19.8 (Tool-result contradiction branch). A tool result that contradicts the final answer should dominate risk. Define

$$g_{\text{tool_contr}} = \max_{e \in \text{tool results}} \{\ell_{\text{contradict}}(e, y) + \ell_{\text{importance}}(e)\}. \quad (19.8)$$

Use for code execution, calculators, databases, and retrieval APIs. If high, require answer revision or explicit uncertainty.

Objective 19.9 (Style-content disentanglement). Keep factual labels invariant under pure style transformations and keep style labels sensitive to style transformations. For a style transformation S , use

$$\mathcal{L} = \|O_{\text{fact}}(\tau) - O_{\text{fact}}(S\tau)\|^2 + \max(0, \gamma - \|O_{\text{style}}(\tau) - O_{\text{style}}(S\tau)\|)^2. \quad (19.9)$$

This prevents the oracle from confusing polished writing with truth.

Objective 19.10 (Bits-per-byte rollback sentinel). Train a small sentinel to predict when oracle-assisted training is beginning to hurt held-out bits-per-byte. The sentinel observes recent validation deltas and oracle action rates:

$$\hat{r}_t = f(\Delta \text{bits-per-byte}_{t-k:t}, \Delta A_{t-k:t}, \text{action_hist}_{t-k:t}). \quad (19.10)$$

If $\hat{r}_t$ exceeds threshold, freeze oracle actions and run rollback evaluation.

20 Safety and limitations

Oracle labels are not truth. The oracle can be biased, miscalibrated, overfit, or vulnerable to distribution shift. High-risk labels should trigger verification or escalation, not automatic irreversible actions.

Do not optimize style over accuracy. Personality and tone controls are useful only when they preserve truth, user intent, and safety. A fluent hallucination is not an improvement. A polite unsafe answer is not an improvement. A concise answer that omits necessary caveats can be worse than a longer answer.

Avoid hidden reasoning exposure. The oracle can read hidden summaries and certificates during training, but user-visible explanations should use concise rationales, citations, verifier results, and uncertainty statements. The system should not expose private hidden chain-of-thought text.

Deployment costs matter. A full oracle transformer may be scientifically interesting and practically useless if it adds latency, artifact bytes, or false security. The first deployable component should usually be a small calibrated head or retrieval gate.

21 Reproducibility checklist

A TropicalGT IV experiment should report:

- base model, checkpoint, tokenizer, context budget, and tropical-attention configuration;
- exact trajectory packet schema and which hidden fields are stored;
- property taxonomy, poset edges, incompatibility nonfaces, ordinal thresholds, and label instructions;

- label sources, human calibration protocol, weak-label functions, and synthetic perturbation generator;
- oracle architecture, head ranks, calibration method, loss weights, and optimization schedule;
- privacy filters and whether raw chain-of-thought text is stored, summarized, or excluded;
- evaluation tasks, held-out split, leakage controls, and confidence intervals;
- downstream model-training actions enabled by the oracle;
- artifact-byte and latency accounting;
- ablation ladder and rollback criteria.

22 Conclusion

TropicalGT IV turns behavior and reasoning quality into trajectory typing. The oracle reads the same structured objects developed in TropicalGT I–III: tropical active supports, iterative reasoning packets, context graphs, retrieved memory, tool events, verifier states, and long-context fill policies. It outputs calibrated property distributions and control signals. The mathematical structure is not ornamental. Tropical logits provide active supports and margins; property posets and incompatibility complexes prevent incoherent labels; sheaf gluing enforces local-to-global consistency; min-plus dynamic programming computes conservative control; and information-theoretic utility links oracle signals to held-out bits-per-byte.

The practical rule is strict. A trajectory oracle should be trained because it improves the model: fewer wrong answers, better grounding, better tool use, safer compliance, better tone control, less context waste, and lower held-out bits-per-byte. If it does not do that, it remains an analysis instrument rather than a deployed component. This completes the first four-paper TropicalGT arc: local tropical computation, dynamical tropical reasoning, tropical context retrieval, and tropical oracle control.

A Additional proof details

A.1 Nonexpansiveness of max-plus logits

Let $g(s) = \max_r(b_r + \langle a_r, s \rangle)$ and assume $\|a_r\|_2 \leq L$ for all r . Then

$$|g(s) - g(s')| \leq L \|s - s'\|_2. \quad (\text{A.1})$$

Indeed,

$$g(s) - g(s') \leq \max_r \langle a_r, s - s' \rangle \leq L \|s - s'\|_2, \quad (\text{A.2})$$

and the reverse inequality follows by swapping s, s' . This bound is used in quantization and projection safety analyses.

A.2 Stanley-Reisner-style incompatibility and gradients

The incompatibility penalty uses the same algebraic intuition as a squarefree monomial ideal: forbidden simultaneous property sets are represented by monomials. If a minimal nonface F has all probabilities high, the product $\prod_{p \in F} \hat{y}_p$ is high. If at least one property is low, the penalty is small. The gradient concentrates on labels participating in actual violations, making it more targeted than a dense pairwise penalty over all labels.

A.3 Why tropical aggregation is useful for qualitative labels

Many behavioral labels are not averages. An answer can be unsafe because of one sentence. A proof can be invalid because of one missing lemma. A citation set can be ungrounded because of one central unsupported claim. A tool plan can be invalid because of one unauthorized call. Max-plus aggregation matches these semantics. Ordinary averaging can hide the decisive event.

B Initial experimental plan

A first TropicalGT IV run should avoid over-engineering.

1. Generate 50k–200k trajectories from a TropicalGT III-style agent on math, coding, retrieval, tool-use, memory, and long-context tasks.
2. Label 16 properties: correctness, verifier agreement, grounding, citation support, proof gap, tool necessity, tool schema validity, memory freshness, safe refusal, unsafe compliance, over-refusal, tone match, verbosity match, uncertainty calibration, context waste, and answer readiness.
3. Train a frozen tropical oracle head with rank 8 per property.
4. Audit calibration and active supports. Reject labels whose active supports are nonsensical.
5. Use the oracle for data filtering only. Train a base-model ablation.
6. If verified quality improves and held-out bits-per-byte does not regress, add preference-pair construction.
7. Train a full oracle transformer only for labels that the head cannot predict from summaries.

C Bibliography

References

- [1] Baran Hashemi, Kurt Pasque, Chris Teska, and Ruriko Yoshida. *Tropical Attention: Neural Algorithmic Reasoning for Combinatorial Algorithms*. OpenReview, NeurIPS 2025 poster, 2025.
- [2] Baran Hashemi, Kurt Pasque, Chris Teska, and Ruriko Yoshida. *Tropical Attention: Neural Algorithmic Reasoning for Combinatorial Algorithms*. arXiv:2505.17190, 2025.
- [3] Baran Hashemi, Kurt Pasque, Chris Teska, and Ruriko Yoshida. *Tropical-Attention official code repository*. GitHub, 2025.
- [4] Ilia Itenberg, Grigory Mikhalkin, and Eugenio Shustin. *Tropical Algebraic Geometry*. Oberwolfach Seminars, Birkhauser, second edition, 2009.
- [5] Diane Maclagan and Bernd Sturmfels. *Introduction to Tropical Geometry*. Graduate Studies in Mathematics, American Mathematical Society, 2015.
- [6] Grigory Mikhalkin. Enumerative tropical algebraic geometry in $\mathbb{R}^2$. *Journal of the American Mathematical Society*, 18(2):313–377, 2005.
- [7] Manfred Einsiedler, Mikhail Kapranov, and Douglas Lind. Non-Archimedean amoebas and tropical varieties. *Journal für die reine und angewandte Mathematik*, 601:139–157, 2006.

- [8] Oleg Viro. Gluing of plane real algebraic curves and constructions of curves of degrees 6 and 7. In *Topology*, Leningrad, 1982, Lecture Notes in Mathematics 1060, Springer, 1984.
- [9] Francois Baccelli, Guy Cohen, Geert Jan Olsder, and Jean-Pierre Quadrat. *Synchronization and Linearity: An Algebra for Discrete Event Systems*. Wiley, 1992.
- [10] Peter Butkovic. *Max-linear Systems: Theory and Algorithms*. Springer, 2010.
- [11] Guy Cohen, Stephane Gaubert, and Jean-Pierre Quadrat. Max-plus algebra and system theory: where we are and where to go now. *Annual Reviews in Control*, 23:207–219, 1999.
- [12] Michel Gondran and Michel Minoux. *Graphs, Dioids and Semirings: New Models and Algorithms*. Springer, 2008.
- [13] Model Context Protocol. *Specification 2025-11-25*. modelcontextprotocol.io, 2025.
- [14] Model Context Protocol. *Server tools specification*. modelcontextprotocol.io, 2025.
- [15] Ashish Vaswani et al. Attention is all you need. *NeurIPS*, 2017.
- [16] Petar Velickovic and Charles Blundell. Neural algorithmic reasoning. *Patterns*, 2021.
- [17] Jordan Hoffmann et al. Training compute-optimal large language models. arXiv:2203.15556, 2022.
- [18] Long Ouyang et al. Training language models to follow instructions with human feedback. *NeurIPS*, 2022.
- [19] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *NeurIPS*, 2023.
- [20] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Weinberger. On calibration of modern neural networks. *ICML*, 2017.
- [21] Alexandru Niculescu-Mizil and Rich Caruana. Predicting good probabilities with supervised learning. *ICML*, 2005.
- [22] A. Philip Dawid. Present position and potential developments: Some personal views: Statistical theory: The prequential approach. *Journal of the Royal Statistical Society A*, 147(2):278–292, 1984.
- [23] Peter Grunwald. *The Minimum Description Length Principle*. MIT Press, 2007.
- [24] Thomas Cover and Joy Thomas. *Elements of Information Theory*. Wiley, second edition, 2006.
- [25] Amelie Schreiber. *TropicalGT: Tropical Attention, Polyhedral Reasoning, and Bits-per-byte-Safe Training for Graph-Token Reasoning Agents*. 2026.
- [26] Amelie Schreiber. *TropicalGT II: Tropical Algebraic Geometry, Patchworked Reasoning Dynamics, and Persistent Polyhedral Training for Graph-Token Reasoning Agents*. 2026.
- [27] Amelie Schreiber. *TropicalGT III: Valuated Context Protocols, Tropical Memory Graphs, and Long-Context Retrieval for Graph-Token Reasoning Agents*. 2026.